Lean Paraphrases: Analyzing the Effect of Wording on Autoformalization Output

Leo Zeng
leozeng60@gmail.com

Abstract

Recent research in autoformalization has placed less emphasis on the fact that Large Language Models are sensitive to slight changes in the wording of problems. Our work complements current research in this area by providing insight into the effects of paraphrasing problems on statement autoformalization. We first manually create a dataset of 50 paraphrased problem pairs, and manually check the outputs of model autoformalizations for consistency with the original problem statement. We then extend this study with an automated pipeline that generates and verifies 1000 additional paraphrase pairs drawn from the FormalMATH benchmark, replacing manual correctness checking with a dual-LLM judge. Across both the manual and automated studies, we find that slight rewordings have a significant impact on the structure of the resulting outputs, even if those outputs are accurate and consistent with the natural language problem statement, and that this sensitivity persists at scale. We argue for the need for a system or intermediary language in order to standardize the format of natural language problem statements, resulting in more reliable model autoformalizations. The source code and data can be accessed with this link: `https://github.com/DaVinci12321/Lean_Paraphrases`.

1 Introduction

In recent years, the rapidly increasing capabilities of Large Language Models (LLMs) have resulted in greater research into the areas of autoformalization and automated theorem proving, shortening the gap between natural language proofs and their counterparts in formal languages. Before this "age" of artificial intelligence, several formal languages were already prominent in the mathematical proof community, including Lean (de Moura and Ullrich, 2021), Isabelle (Paulson, 1989), Coq (Coquand and Huet, 1988), and others. These languages allowed for the definitive verification of mathematical proofs, and thus many projects were created and are still being created to formalize the proofs of key theorems, such as Fermat's Last Theorem (Best et al., 2025).

As the reasoning capabilities of LLMs have been increasing, research into the connections between AI and formal math has also increased. For example, autoformalization, which is the translation between natural language mathematical problems and their formal language equivalents, has become more popular, with the goal being the automated translation of proofs into formal languages (Wu et al., 2022). In addition, automated theorem proving has gained popularity as well, with well known models such as AlphaProof (Hubert et al., 2025) being able to solve complex math olympiad problems and even start to work on research-level mathematics through the use of these formal languages.

However, even the best autoformalization models trained on datasets of thousands of problems are not perfect. Indeed, autoformalization models often make tiny mistakes. Strikingly, models are not perfect even at determining whether a formal statement is equivalent to a natural language version of the problem, as shown by the studies in the CriticLean project (Peng et al., 2026).

This sensitivity to input wording is not unique to formal mathematics. Broader natural language processing research has shown that LLMs can be vulnerable to semantically-preserving prompt

Preprint.

perturbations across a wide range of tasks, with such perturbations sometimes even changing which model appears best on a benchmark (Zhu et al., 2023; Romanou et al., 2026). Therefore, it is natural to ask whether the same phenomenon appears in the more precise setting of autoformalization, where one wrong reading of a single word can change the mathematical content of a formal statement entirely.

This area of autoformalization that analyzes how the phrasing of natural language questions can affect an autoformalized output has been relatively overlooked by current research. Nevertheless, it is important to study how slight changes in wording can affect a model's output, since this has critical implications for the overall field of autoformalization.

In this paper we investigate the question of how phrasing affects autoformalization by creating paraphrases of problems and then analyzing LLM formalization outputs. We evaluate not only the effectiveness of current models at autoformalization, but also their sensitivity to changes in wording or rigor. We first conduct a manually constructed and manually verified study of 50 problems, then extend this analysis with an automated pipeline that reproduces the same experimental design on 1000 additional problems, replacing manual paraphrase construction and correctness checking with calibrated LLM judges.

2 Related works

Early research into autoformalization utilized large datasets of formal-informal problem pairs in order to fine-tune LLMs, producing models with state-of-the-art capabilities in autoformalization. However, with the progress in terms of capabilities flattening over time, and with current models being able to formalize the majority of problem statements with few errors, more attention has turned to the field of automated theorem proving. Indeed, progress on automated theorem proving has been a lot more obvious, with models such as AlphaProof (Hubert et al., 2025) and others making huge progress on not only olympiad but also research mathematics.

Despite the continuation of research in the autoformalization area, there has been relatively little attention to the phrasing of natural language statements. This could potentially have a huge impact on autoformalization capabilities, so it is definitely an area worth exploring.

Some recent research has already been done on evaluating the effects of paraphrasing problems. There has been work that has shown that AI models are quite sensitive to slight rewordings of mathematical proofs in large formal-informal problem pair datasets (Gui et al., 2026). In addition, research has already shown that LLMs can change their outputs substantially if problem statements are marginally paraphrased (Moore and Shah, 2025). More broadly, work on general-purpose LLM robustness has found that model outputs, and even the relative ranking of models on a benchmark, can shift substantially under semantically-equivalent prompt rewordings outside of mathematical domains (Zhu et al., 2023; Romanou et al., 2026), suggesting that the wording sensitivity observed in autoformalization may be a specific instance of a more general property of LLMs.

However, the research that has been done before has either analyzed only the rewording of mathematical *proofs*, or has relied on AI models themselves to reword problems. Thus our research contributes more to this area by analyzing the rewordings of mathematical *statements* and by manually generating paraphrases in order to create a more controlled and standardized environment. We further extend this contribution with a calibrated, automated pipeline that reproduces the same controlled paraphrase-pair design at a larger scale, drawing source problems from the FormalMATH benchmark (Yu et al., 2025) and replacing manual human judgment with a dual-LLM judge.

3 Dataset construction and model experimentation

3.1 Manual study

3.1.1 Data construction

We created a dataset of 50 problems, each with 2 paraphrases. To select the problems, we looked through datasets of formal-informal pairs of problems, such as Lean Workbook (Ying et al., 2024), Constructive Bench (Sun et al., 2025), and Prover Bench (Xin et al., 2024). After selecting 50 problems that had potential to be paraphrased, we manually created paraphrases for each problem.

In addition to extracting the informal problem statements from the dataset, we also extracted their formal counterparts as a reference to be able to compare with the AI model outputs. The problem statements were all stored in their LaTeX forms, which is what was ultimately fed to the models.

Each of the 50 problems was paraphrased into two different versions: one version that was more rigorous and mathematically specific, relying more on symbols to explain the problem, and one version that was less specific, relying more on words to explain the problem. In this way, we were able to test the AI models' capabilities in translating wordy problems that were less rigorous in terms of their mathematical definitions and equations.

During the process of paraphrasing a problem to become less rigorous, we essentially replaced symbolic equations and definitions with worded descriptions. The goal of each non-rigorous paraphrase was to include as few symbols and numbers as possible. This allowed us to test the models' abilities to synthesize worded descriptions into symbolic equations. For example, below are the two paraphrases of a problem:

Paraphrase 1 (Rigorous). For $a, b, c, d > 0$ and $abcd = 1$, prove that $\frac{1}{1+(1+a)^2} + \frac{1}{1+(1+b)^2} + \frac{1}{1+(1+c)^2} + \frac{1}{1+(1+d)^2} \leq \frac{4}{5}$.

Paraphrase 2 (Non-rigorous). For positive numbers a, b, c, d whose product is 1, prove that the cyclic sum of $\frac{1}{1+(1+a)^2}$ is at most $\frac{4}{5}$.

By using worded descriptions such as "cyclic sum", "complex conjugate", "central binomial coefficient", and others, we could use the non-rigorous paraphrases to evaluate the models' strengths and weaknesses in terms of interpreting these simplifications. Manually constructing the dataset also ensures the quality of our input data, along with allowing us to control the difference in rigor between the two types of paraphrases.

3.1.2 Evaluation

We ran our manually created dataset (all 100 problems) through two of the most recently released models: OpenAI's GPT 5.6 Sol and Anthropic's Claude Opus 5. In this way, we were able to test the capabilities of the current state-of-the-art models when autoformalizing paraphrases of the same problem.

We used the following prompt for each of the models:
Prompt: Formalize this natural language problem into Lean 4 language. Only create the prompt, and use 'sorry' as the proof. Your response should only include the final formalized Lean 4 statement.

For each of the 100 problems, the model was given the above prompt along with the LaTeX form of the problem, without any additional context. Moreover, paraphrases of the same problem were tested independently of each other, in order to create a more standardized environment for the model, with no interference from other problems or paraphrases.

Note that for the manual testing phase we were only analyzing the models' capabilities at one-shot autoformalization, and so there was no opportunity for the models to correct any of their errors, or to try to get a correct answer in k passes. Additionally, we were only testing the autoformalization skills of the models for problem statements, and so we did not add any of the solutions, thus requiring the model to output a 'sorry' as a placeholder for the formalized solution.

With the results that the two LLMs gave, we manually checked each autoformalization for consistency with the problem statement, and then also compared each problem with its paraphrased variant, in order to find the subtle differences in autoformalization when the problem is phrased slightly differently. Thus the correctness of each autoformalization was only evaluated based on consistency with the natural language problem, and did not include requirements for correct compilation. Obviously general Lean 4 syntax was checked, but a more thorough and rigorous review was not done. The purpose of the manual testing was to more precisely analyze the effect of paraphrasing on model output, so there was not a huge emphasis placed on consistency with Lean 4 syntax.

3.2 Automated study

3.2.1 Data construction

The manual dataset of Section 3.1.1 is deliberately small, since every pair had to be handwritten and every autoformalization had to be checked by hand. To test whether our findings generalize beyond 50 hand-picked problems, we built an automated pipeline that reproduces the same rigorous/non-rigorous paraphrase design at a much larger scale, drawing source problems from FormalMATH (Yu et al., 2025), a benchmark of thousands of formally verified Lean 4 problems spanning high-school olympiad through undergraduate-level mathematics.

For each candidate source problem, Claude Opus 5 generates a rigorous and a non-rigorous paraphrase, using the 50 manual pairs from Section 3.1.1 as examples of the desired style, and using the problem's formalized statement as a reference during analysis. The generated pair is then scored by two independent judge models, Claude Opus 5 and GPT 5.6 Sol, each of which rates four claims on a 0 to 1 confidence scale: that the rigorous paraphrase is equivalent to the source problem, that the non-rigorous paraphrase is equivalent to the source problem, that the two paraphrases are equivalent to each other, and that the intended contrast between them (rigorous versus non-rigorous) is genuine. A pair is only accepted into the dataset once every claim from *both* judges is affirmed and every confidence score clears a threshold of 0.85; otherwise the pair is regenerated with the judges' specific objections attached, for up to three attempts. Source problems whose reference Lean statement defines helper declarations before the main theorem were excluded, since they cannot be reduced to the single self-contained statement our pipeline requires for automated comparison.

Running this pipeline against FormalMATH yielded 1000 accepted paraphrase pairs (2000 problem statements total) out of 1203 candidate problems attempted, an 83.1% acceptance rate. Together with the 50 manually written pairs, this gives us an automated extension roughly twenty times larger than the original manual dataset, while preserving the same controlled rigorous/non-rigorous design.

3.2.2 Evaluation

We ran all 2000 LaTeX problem statements from the 1000 automatically-generated pairs through the same two models, GPT 5.6 Sol and Claude Opus 5, using the identical one-shot prompt from Section 3.1.2. As in the manual study, paraphrases of the same problem were formalized independently, models were only given a single attempt, and only the formalized statement (and not a full proof) was required.

In place of manual review, each of the 2000 resulting autoformalizations was judged automatically for equivalence with its natural-language paraphrase by the same two-judge procedure used to build the dataset: both Claude Opus 5 and GPT 5.6 Sol independently reviewed the paraphrase, the corresponding formalization, and the reference FormalMATH Lean statement as context (explicitly not an answer key, since a correct formalization that differs from the reference but still matches the natural language statement is counted as equivalent), and each judge returned a verdict with a confidence score. A problem is labeled `EQUIVALENT` or `NOT_EQUIVALENT` only when both judges agree and at least one clears a confidence threshold of 0.80; otherwise it is labeled `UNDECIDED`. To keep the results free of the disagreement cases a human would need to resolve, we pruned any of the 1000 problems for which *any* of its four judged configurations (two models each with two paraphrase types) came back `UNDECIDED`, which removed 110 problems and left a set of 890 problems (1780 judged autoformalizations) for the analysis in Section 4.2.

4 Results and analysis

4.1 Manual study results

Table 1: Number of correct autoformalizations by model (manual study, 100 problems)

Model	Rigorous Paraphrase	Non-rigorous Paraphrase	Total
GPT 5.6 Sol	48/50	47/50	95/100
Claude Opus 5	47/50	46/50	93/100

For the manual study, the GPT model got 95% of the autoformalizations correct, while the Claude model only got 93% of the autoformalizations correct. Both models were more likely to get the autoformalizations for both paraphrases of a single problem wrong, rather than getting only one paraphrase of a problem wrong. The two models were also somewhat consistent in getting the same problem autoformalizations wrong.

As an example, both models got the autoformalizations wrong for both paraphrases of these two problems (the full Lean formalizations appear in Appendix A.1 and Appendix A.2):

Problem 1 (Manual Study), Rigorous Paraphrase. Prove that $a + b + c \geq \frac{a^3+b^3+c^3}{a^2+b^2+c^2}$.

Problem 1 (Manual Study), Non-rigorous Paraphrase. Prove that the sum of three numbers is at least the quotient between the sum of the cubes of the three numbers and the sum of the squares of the three numbers.

Problem 2 (Manual Study), Rigorous Paraphrase. Prove that $\frac{a^2+b^2+c^2}{a+b+c} \geq 1$.

Problem 2 (Manual Study), Non-rigorous Paraphrase. Prove that the quotient between the sum of the squares of three numbers and the sum of those three numbers is always at least 1.

In both of these problems, the two models made some new assumptions in the formalization that were never stated in the natural language problems. Both GPT and Claude assumed in one of their formalizations that the numbers a, b, c would be natural numbers. In another case they both assumed that the three numbers would be positive. However, these conditions were never a part of the problem. One possible explanation would be that in the problems above, the variables a, b, c were described as "numbers", which could have been interpreted by the LLMs to mean integers, or positive numbers. This was a recurring problem for Claude, since it got another paraphrased problem's autoformalization wrong due to this assumption that a variable, described as a "number", would be an integer.

Although the models' autoformalizations for the two paraphrases of a problem were often quite similar and almost identical, there were a few cases in which their results reflected the natural language inputs. One of the clearest ways this happened was through variable names. The GPT model in particular showed this often: its autoformalizations of non-rigorous paraphrases often used descriptive variable names such as "largest", "smallest", "first", "second", "third", etc., which directly reflected the descriptions actually used in the problem. However, in the corresponding, more rigorous paraphrase, this did not happen, as the variables were all named a, b, c, or x, y, z. In this way, the models also copied specific parts of a less rigorous paraphrase, in ways that were often less simple than if they had been looking at a more rigorous one. As another example, in one problem the amount 25 percent was mentioned, and the LLMs formalized it as $\frac{25}{100}$, instead of the simplified $\frac{1}{4}$ which was used in the more rigorous paraphrasing.

This clearly shows that the phrasing of a natural language problem largely affects the resulting autoformalization. For instance, a rigorous paraphrase with "$x \equiv 0 \pmod 2$" and the matching non-rigorous paraphrase with "x is even" have autoformalizations that reflect their respective inputs. In another case, the differences in the autoformalizations of the two paraphrases of a problem were also clearly the result of the phrasing differences: "$a = 2^n 3^m$" in the rigorous paraphrase and "a is a number with only 2 and 3 as its prime divisors" in the non-rigorous paraphrase.

Overall, both of the models achieved a high accuracy on the dataset, although they occasionally made tiny mistakes. Their results also showed the influence of the paraphrased problem statements, with two autoformalizations of the same problem often being quite different due to the changes in wording between the two paraphrases.

4.2 Automated study results

Table 2: Equivalence rate by model and configuration (automated study, 890 problems, dual-LLM judges)

Configuration	Equivalent	Not Equivalent	Overall Equivalence Rate
GPT 5.6 Sol, Rigorous	875/890	15/890	98.3%
GPT 5.6 Sol, Non-rigorous	867/890	23/890	97.4%
Claude Opus 5, Rigorous	871/890	19/890	97.9%
Claude Opus 5, Non-rigorous	865/890	25/890	97.2%

The automated pipeline roughly reproduces the results of the manual study at a larger scale. Again, both models autoformalize a large majority of problems correctly. Additionally, both do slightly worse on the non-rigorous paraphrases than on the corresponding rigorous ones (GPT: 98.3% vs. 97.4%; Claude: 97.9% vs. 97.2%), which supports the manual study's finding that rewording problem statements to be less rigorous makes it somewhat harder to formalize the statements correctly.

Of the 890 problems, GPT's verdict flipped between its rigorous and non-rigorous autoformalization on 26 problems (2.9%), and Claude's flipped on 22 problems (2.5%). In the large majority of remaining cases the model was consistently correct on both paraphrases (GPT: 858/890; Claude: 857/890) rather than consistently wrong (GPT: 6/890; Claude: 11/890). In addition, the two judge models also largely agreed with each other, suggesting that the errors induced by non-rigorous phrasing reflect a genuine ambiguity or difficulty in the paraphrase itself.

As with the manual study, individual problems can show the effect of rewording problem statements. On one problem, the rewording changed the actual mathematical content of GPT's formalization (Appendix A.3):

Problem 1 (Automated Study), Rigorous Paraphrase. Let $f : \mathbb{R} \to \mathbb{R}$; prove that $f(xy+f(x^2)) = xf(x+y)$ holds for all $x, y \in \mathbb{R}$ if and only if for every $x \in \mathbb{R}$ one has $f(x) = 0$ or $f(x) = x$.

Problem 1 (Automated Study), Non-rigorous Paraphrase. Given a real-valued function of a real variable, prove that it has the property that, for every choice of a first and a second real number, its value at the product of the two numbers increased by its value at the square of the first number equals the first number times its value at the sum of the two numbers, exactly when, for every real number, the function either vanishes there or returns that same number.

GPT's formalization of the rigorous paraphrase correctly nested the inner term inside the outer function's argument, matching $f(xy + f(x^2))$. In the non-rigorous paraphrase, however, the phrase "its value at the product of the two numbers increased by its value at the square of the first number" was somewhat ambiguous about where "increased by" attaches: GPT's formalization moved the addition outside the outer function entirely, producing $f(xy) + f(x^2) = xf(x + y)$, a different functional equation than the one asked about. The automated judges accordingly marked the non-rigorous formalization as not equivalent to its source problem, showing that a formalization may be syntactically correct but not semantically equivalent.

On another problem, the two models made identical logic errors due to rewordings (Appendix A.4).

Problem 2 (Automated Study), Rigorous Paraphrase. Let $f : \mathbb{R} \to \mathbb{R}$; prove that $f(xf(x) + f(y)) = f(x)^2 + y$ holds for all $x, y \in \mathbb{R}$ if and only if for every $x \in \mathbb{R}$ one has $f(x) = x$ or $f(x) = -x$.

Problem 2 (Automated Study), Non-rigorous Paraphrase. Let f be a real-valued function of a real variable; prove that f sends the sum of f of y and the product of x with f of x to the sum of y and the square of the value of f at x, for every choice of real numbers x and y, exactly when at each real number the function returns either that number itself or its negative.

On this problem, both GPT and Claude made the same mistake on the rigorous paraphrase: both models formalized the phrase "for every $x \in \mathbb{R}$ one has $f(x) = x$ or $f(x) = -x$" as $(\forall x, f(x) = x) \vee (\forall x, f(x) = -x)$ instead of the intended condition $\forall x, (f(x) = x \vee f(x) = -x)$. The non-rigorous paraphrase shows the distinction more clearly with "either...or...", and thus both models formalized the non-rigorous paraphrase correctly. Because this same error appears identically in two

different models on the same wording, it shows another example of ambiguous wording, this time in the rigorous paraphrase, that causes confusion in the autoformalizing models.

5 Discussion

In this project both models performed quite strongly in terms of the accuracy of their formalizations. Despite the non-rigorous paraphrases being somewhat vague in their descriptions of the problems, the models were still able to formalize them with an accuracy comparable to that of their formalizations of rigorous paraphrases. Despite that, the models still showed that they are not infallible, making mistakes such as domain errors and extra hypotheses.

As seen from the results of these experiments, LLMs are still quite sensitive to small changes in the phrasing of problems. When expanding rigorous mathematical symbols into more wordy, non-rigorous descriptions, some of the overall meaning is lost, and this in turn influences the models' interpretations of the problem.

The phrasing of problems matters, even if a model is still able to correctly autoformalize that problem. As the results show, the difference between writing "even" and writing "$0 \pmod 2$" in a problem is clearly reflected in the model's autoformalization. Thus, future autoformalization research should place some emphasis on the quality of the natural language data used. In the field of autoformalization, it would also be beneficial to have a "middle ground" between natural and formal languages, to facilitate the translation between the two.

Since some of the most obvious errors that the models made during autoformalization were domain errors, this intermediate, semi-formal language would be able to clear up any vagueness of the conditions of each problem, allowing models to more accurately formalize the problems. For instance, if a natural language problem states some properties of "numbers", then it would be useful to have a model that could first translate the problem into the intermediate language, to make clear whether "numbers" refers to real numbers, or integers, or natural numbers, etc.

Although this intermediate language would share many similarities with the actual formal language, it would keep many of the properties of natural language, with an English (or other world language) wording. Having this intermediate language would not only be beneficial for autoformalization, but also for training models to perform autoformalization tasks. Translating from natural language to the intermediate language and then from the intermediate language to formal language would presumably be easier as separate tasks. We would effectively be splitting up a more complex task of autoformalization into two simpler parts, and models trained on these two tasks would be much more accurate than any single model translating directly from natural to formal language.

The automated extension of Section 4.2 strengthens this argument. The single biggest objection to the manual study's conclusions was that they were drawn from only 100 problems, so any given flip could possibly have been an outlier. At 890 problems drawn from an entirely different source benchmark, both autoformalizing models still flip their verdict on a non-trivial fraction of problems purely as a result of rewording (2.5–2.9%). A rewording sensitivity of a few percent may sound small, but for an autoformalization pipeline meant to feed an automated theorem prover, a few percent of mistranslated theorems is exactly the kind of error that an intermediate, disambiguating language could catch before it propagates further.

6 Limitations

Firstly, there are some limitations to the findings of this paper with regard to the accuracy of these models in autoformalization. Giving models only one attempt at autoformalization obviously decreases their accuracy compared with a pass@k experiment. This applies equally to the manual and automated studies, since the automated pipeline reuses the manual study's one-shot prompt verbatim. In addition, the manual dataset (being only 50 problems with 2 paraphrases each, i.e. 100 total problems) limits the breadth and generalization of the manually-verified findings on its own; the automated extension in Section 3.2.1 addresses this specific limitation by scaling to 890 judge-agreed problems, but only for the two models used as autoformalizers throughout this paper. With this research only analyzing the performance of two models, its findings are again less generalizable,

with the need for analysis of more state-of-the-art models in order to reveal the current capabilities of LLMs in the field of autoformalization.

The automated extension also introduces limitations of its own. First, replacing manual human judgment with a dual-LLM judge procedure means that the correctness and paraphrase-sensitivity statistics in Section 4.2 are only as reliable as the judges themselves; we mitigated this by requiring agreement between two different model families and a high confidence threshold, and by discarding rather than resolving the cases where the judges disagreed, but a systematic blind spot shared by both judge models would not be caught by this procedure. Second, that same pruning step removes precisely the problems closest to the decision boundary, which are plausibly the most paraphrase-sensitive ones; the pre-pruning statistics in Section 4.2 suggest this makes our headline flip rates a conservative underestimate rather than an inflated one, but it does mean the 890-problem set is not a uniformly random sample of the original 1000. Third, the automated dataset draws exclusively from FormalMATH, a single source benchmark, whereas the manual dataset drew from three; consistent findings across both suggest the phenomenon is not an artifact of any one source, but the automated results alone would not show this. Finally, both the paraphrase generator and both correctness/equivalence judges are themselves drawn from the same two model families being evaluated as autoformalizers, so some caution is warranted in how independent these roles really are, even though generation, formalization, and judging are separate prompts and, in the case of GPT 5.6 Sol, a separate model family from the generator.

References

Best, A. J., Birkbeck, C., Brasca, R., Rodriguez Boidi, E., van de Velde, R., and Yang, A. (2025). A complete formalization of fermat's last theorem for regular primes in lean. *Annals of Formalized Mathematics*, 1:103–132.

Coquand, T. and Huet, G. (1988). The calculus of constructions. *Information and Computation*, 76(2–3):95–120.

de Moura, L. and Ullrich, S. (2021). The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, Cham. Springer.

Gui, Z., Yang, S., and Shi, Z. (2026). Evaluating the robustness of proof autoformalization in lean 4. *arXiv preprint arXiv:2606.14867*.

Hubert, T., Mehta, R., Sartran, L., Horváth, M. Z., Žužić, G., Wieser, E., Huang, A., Schrittwieser, J., Schroecker, Y., Masoom, H., Bertolli, O., Zahavy, T., Mandhane, A., Yung, J., Beloshapka, I., Ibarz, B., Veeriah, V., Yu, L., Nash, O., Lezeau, P., Mercuri, S., Sönne, C., Mehta, B., Davies, A., Zheng, D., Pedregosa, F., Li, Y., von Glehn, I., Rowland, M., Albanie, S., Velingker, A., Schmitt, S., Lockhart, E., Hughes, E., Michalewski, H., Sonnerat, N., Hassabis, D., Kohli, P., and Silver, D. (2025). Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613.

Moore, H. and Shah, A. (2025). Evaluating autoformalization robustness via semantically similar paraphrasing. *arXiv preprint arXiv:2511.12784*.

Paulson, L. C. (1989). The foundation of a generic theorem prover. *Journal of Automated Reasoning*, 5:363–397.

Peng, Z., Yao, Y., Ma, K., Guo, S., Li, Y., Zhang, Y., Zhang, C., Zhang, Y., Yu, Z., Li, L., Liu, M., Xia, Y., Shen, J., Wu, Y., Cao, Y., Zhang, Z., Huang, W., Liu, J., and Zhang, G. (2026). Criticlean: Critic-guided reinforcement learning for mathematical formalization. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 3049–3088. Association for Computational Linguistics.

Romanou, A., Ibrahim, M., Ross, C., Shaib, C., Oktar, K., Bell, S. J., Ovalle, A., Dodge, J., Bosselut, A., Sinha, K., and Williams, A. (2026). Brittlebench: Quantifying llm robustness via prompt sensitivity. *arXiv preprint arXiv:2603.13285*.

Sun, J., Tang, Y., Li, A., Maddison, C. J., and Meel, K. S. (2025). Enumerate-conjecture-prove: Formally solving answer-construction problems in math competitions. *arXiv preprint arXiv:2505.18492*.

Wu, Y., Jiang, A. Q., Li, W., Rabe, M. N., Staats, C., Jamnik, M., and Szegedy, C. (2022). Autoformalization with large language models. In *Advances in Neural Information Processing Systems*.

Xin, H., Guo, D., Shao, Z., Ren, Z., Zhu, Q., Liu, B., Ruan, C., Li, W., and Liang, X. (2024). Deepseek-prover: Advancing theorem proving in llms through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*.

Ying, H., Wu, Z., Geng, Y., Wang, J., Lin, D., and Chen, K. (2024). Lean workbook: A large-scale lean problem set formalized from natural language math problems.

Yu, Z., Peng, R., Ding, K., Li, Y., Peng, Z., Liu, M., Zhang, Y., Yuan, Z., Xin, H., Huang, W., Wen, Y., Zhang, G., and Liu, W. (2025). Formalmath: Benchmarking formal mathematical reasoning of large language models. *arXiv preprint arXiv:2505.02735*.

Zhu, K., Wang, J., Zhou, J., Wang, Z., Chen, H., Wang, Y., Yang, L., Ye, W., Zhang, Y., Gong, N. Z., and Xie, X. (2023). Promptbench: Towards evaluating the robustness of large language models on adversarial prompts. *arXiv preprint arXiv:2306.04528*.

A Formalized problem statements

This appendix gives the full Lean 4 formalizations underlying the paraphrase-pair examples discussed in Section 4.1 and Section 4.2. Every statement uses `sorry` in place of a proof, since only the formalized statement, not the proof, was evaluated in this paper.

A.1 Manual study, problem 1 formalizations

GPT 5.6 Sol, rigorous paraphrase.

```
theorem sum_ge_sum_cubes_div_sum_squares
   (a b c : ℝ) (ha : 0 ≤ a) (hb : 0 ≤ b) (hc : 0 ≤ c) :
   a + b + c ≥ (a ^ 3 + b ^ 3 + c ^ 3) / (a ^ 2 + b ^ 2 + c ^ 2) := by
 sorry
```

GPT 5.6 Sol, non-rigorous paraphrase.

```
theorem sum_ge_sum_cubes_div_sum_squares
   (a b c : ℝ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
   a + b + c ≥ (a ^ 3 + b ^ 3 + c ^ 3) / (a ^ 2 + b ^ 2 + c ^ 2) := by
 sorry
```

Claude Opus 5, rigorous paraphrase.

```
theorem sum_ge_cube_sum_div_sq_sum
   (a b c : ℝ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
   a + b + c ≥ (a ^ 3 + b ^ 3 + c ^ 3) / (a ^ 2 + b ^ 2 + c ^ 2) := by
 sorry
```

Claude Opus 5, non-rigorous paraphrase.

```
theorem sum_ge_cubes_div_squares
   (a b c : ℝ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
   (a^3 + b^3 + c^3) / (a^2 + b^2 + c^2) ≤ a + b + c := by
 sorry
```

A.2 Manual study, problem 2 formalizations

GPT 5.6 Sol, rigorous paraphrase.

```
theorem sum_squares_div_sum_ge_one
   (a b c : ℕ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
   ((a : ℝ) ^ 2 + (b : ℝ) ^ 2 + (c : ℝ) ^ 2) /
       ((a : ℝ) + (b : ℝ) + (c : ℝ)) ≥ 1 := by
 sorry
```

GPT 5.6 Sol, non-rigorous paraphrase.

```
theorem sum_squares_div_sum_ge_one (a b c : ℕ) (h : 0 < a + b + c) :
   1 ≤ (a ^ 2 + b ^ 2 + c ^ 2) / (a + b + c) := by
 sorry
```

Claude Opus 5, rigorous paraphrase.

```
theorem sq_sum_div_sum_ge_one
   (a b c : ℝ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c)
   (habc : a + b + c = 3) :
   (a ^ 2 + b ^ 2 + c ^ 2) / (a + b + c) ≥ 1 := by
 sorry
```

Claude Opus 5, non-rigorous paraphrase.

```
theorem sum_sq_div_sum_ge_one
```

```
  (a b c : ℕ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
  1 ≤ ((a : ℝ) ^ 2 + (b : ℝ) ^ 2 + (c : ℝ) ^ 2) /
      ((a : ℝ) + (b : ℝ) + (c : ℝ)) := by
sorry
```

A.3 Automated study, problem 1 formalizations

GPT 5.6 Sol, rigorous paraphrase.

```
theorem functional_equation_iff (f : ℝ → ℝ) :
  (∀ x y : ℝ, f (x * y + f (x ^ 2)) = x * f (x + y)) ↔
    (∀ x : ℝ, f x = 0 ∨ f x = x) := by sorry
```

GPT 5.6 Sol, non-rigorous paraphrase.

```
theorem functional_equation_characterization (f : ℝ → ℝ) :
  (∀ x y : ℝ, f (x * y) + f (x ^ 2) = x * f (x + y)) ↔
    ∀ x : ℝ, f x = 0 ∨ f x = x := by sorry
```

A.4 Automated study, problem 2 formalizations

GPT 5.6 Sol, rigorous paraphrase.

```
theorem functional_equation_iff (f : ℝ → ℝ) :
  (∀ x y : ℝ, f (x * f x + f y) = (f x) ^ 2 + y) ↔
    (∀ x : ℝ, f x = x) ∨ (∀ x : ℝ, f x = -x) := by sorry
```

GPT 5.6 Sol, non-rigorous paraphrase.

```
theorem functional_equation_iff (f : ℝ → ℝ) :
  (∀ x y : ℝ, f (f y + x * f x) = y + (f x) ^ 2) ↔
    (∀ x : ℝ, f x = x ∨ f x = -x) := by sorry
```

Claude Opus 5, rigorous paraphrase.

```
theorem f_eq_iff (f : ℝ → ℝ) :
  (∀ x y : ℝ, f (x * f x + f y) = (f x) ^ 2 + y) ↔
    ((∀ x : ℝ, f x = x) ∨ (∀ x : ℝ, f x = -x)) := by sorry
```

Claude Opus 5, non-rigorous paraphrase.

```
theorem f_eq_iff (f : ℝ → ℝ) :
  (∀ x y : ℝ, f (f y + x * f x) = y + (f x) ^ 2) ↔
    (∀ x : ℝ, f x = x ∨ f x = -x) := by sorry
```